

```
In [ ]: import os
import time
import sys

import marimo as mo

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from nilearn.glm.first_level import FirstLevelModel, make_first_level_design_matrix
from nilearn.interfaces.fmriprep import load_confounds
from nilearn.image import load_img, smooth_img, math_img
from nilearn.maskers import NiftiMasker, NiftiLabelsMasker
from nilearn.plotting import plot_design_matrix, plot_roi, view_img, plot_stat_map

import nilearn
print(f"nilearn version: {nilearn.__version__}")
```

nilearn version: 0.11.1

Set up directories and hyperparameters

```
In [ ]: # Hyperparameters
TR = 2.0
FWHM = 6.0
N_SCANS = 182

SEED_LABEL = 37

num_runs = 3
runs = [str(i + 1).zfill(2) for i in range(num_runs)]
```

```
In [ ]: # directories
current_dir = os.getcwd()
print(f"Current directory: {current_dir}")

# on HPC or on local machine?

# if HPC, get sub ID from command line args
if "dartfs" in current_dir:
    print("\nRunning on HPC")
    sub = sys.argv[1]
    sub = sub.zfill(2) # zero-pad to 2 digits if needed

    data_dir = f"/dartfs/rc/lab/D/DBIC/DBIC/psych160/data/stopsignal/derivatives"
    mask_dir = f"/dartfs/rc/lab/D/DBIC/DBIC/psych160/masks/k50_2mm.nii.gz"

    # sleep for 10s if not sub-01 - for creating save dirs on HPC
    if sub != "01":
        time.sleep(10)

# if local, manually set sub ID here
```

```

else:
    print("\nRunning locally")
    sub = "01"

    data_dir = f"../data/fmriprep/sub-{sub}/func"
    mask_dir = f"../data/masks/k50_2mm.nii.gz"

    print(f"\nRunning sub-{sub}")
    print(f"\nHyperparameters: TR={TR}, FWHM={FWHM}, runs={runs}")

```

Current directory: /Users/gabe/Downloads/exam/connectivity

Running locally

Running sub-01

Hyperparameters: TR=2.0, FWHM=6.0, runs=['01', '02', '03']

```

In [ ]: # set and create save directories
        save_dir = f"{current_dir}/output/sub-{sub}"
        os.makedirs(save_dir, exist_ok=True)

```

Load the fMRI data for this sub

```

In [ ]: fmri_imgs = []
        events = []
        confounds = []

        for run in runs:

            if sub == "12" and run == "03":
                print(f"Skipping sub-{sub} run-{run} due to missing run.")
                continue

            events_df = pd.read_csv(f"{data_dir}/sub-{sub}_task-stopsignal_run-{run}")
            events_df = events_df.dropna(subset=['duration']) # remove 1st col, has
            events_df['trial_type'] = events_df['trial_type'].str.replace(' ', '_')
            events_df = events_df[["onset", "duration", "trial_type"]] # keep only r
            events.append(events_df)

            fmri_img_path = f"{data_dir}/sub-{sub}_task-stopsignal_run-{run}_space-M
            _fmri_img = load_img(fmri_img_path)
            _fmri_img = smooth_img(_fmri_img, fwhm=FWHM) # apply smoothing
            fmri_imgs.append(_fmri_img)

            confound, _ = load_confounds(fmri_img_path,
                                        strategy=["motion", "wm_csf", "compcor", "h
                                        motion="full",
                                        wm_csf="basic")

            confounds.append(confound)

        assert len(events) == len(fmri_imgs) == len(confounds), f"Mismatch in events
        print(f"Loaded {len(events)} runs of data for sub-{sub}.")

```

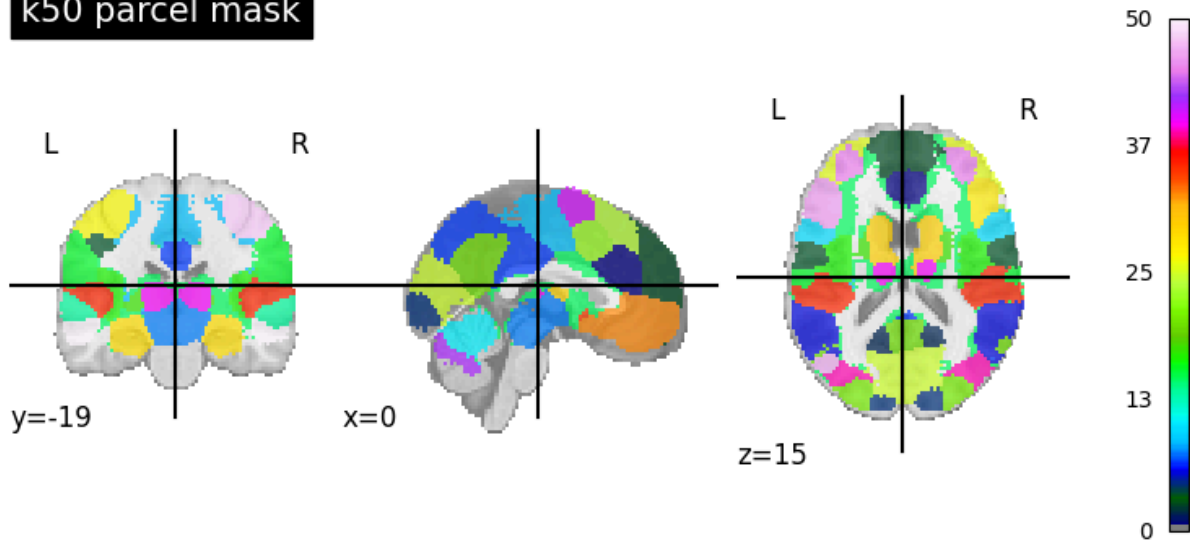
Loaded 3 runs of data for sub-01.

Load parcel mask

```
In [ ]: parcel_mask = load_img(mask_dir)

if "dartfs" not in current_dir:
    plot_roi(parcel_mask, title="k50 parcel mask", colorbar=True)
    plt.show()
```

k50 parcel mask



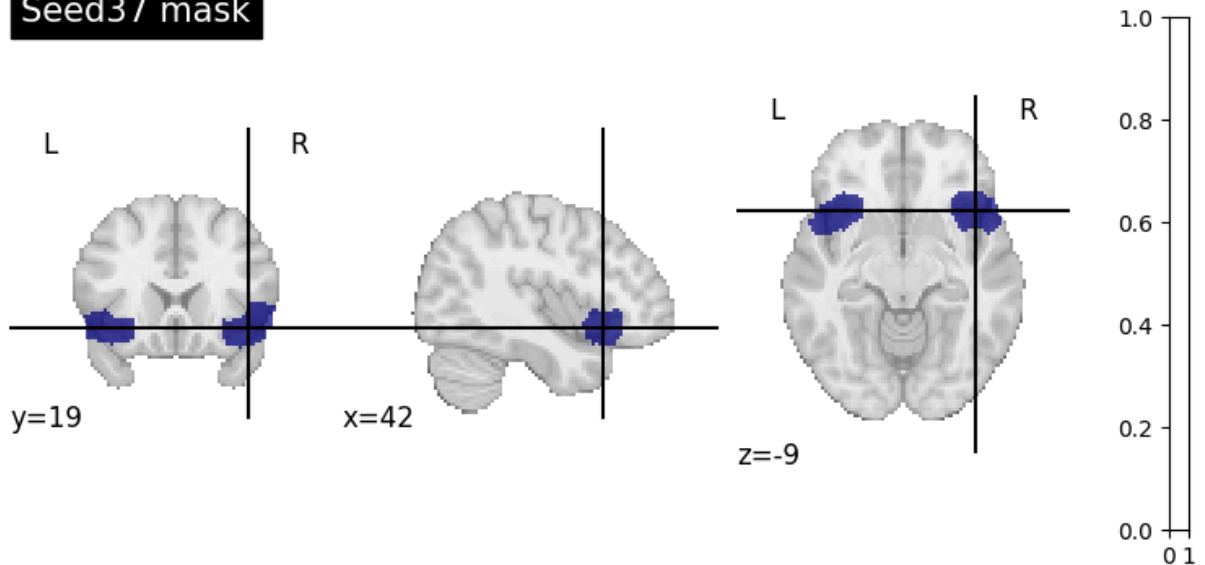
```
In [ ]: # confirm parcel mask has same shape and affine as fMRI data
assert parcel_mask.shape == fmri_imgs[0].shape[:3], f"Parcel mask shape {par
assert np.allclose(parcel_mask.affine, fmri_imgs[0].affine), "Parcel mask af
print("Parcel mask shape and affine match fMRI images.")
```

Parcel mask shape and affine match fMRI images.

```
In [ ]: # create binary mask for seed region
# need to add 1 to SEED_LABEL because parcel labels are 1-indexed, but SEED_
mask_img = math_img(
    f"np.where(np.round(img) == {SEED_LABEL + 1}, 1, 0)", img=parcel
)
if "dartfs" not in current_dir:
    plot_roi(mask_img, title=f"Seed{SEED_LABEL} mask", colorbar=True)
    plt.show()
```

```
/Users/gabe/.cache/uv/archive-v0/MLTFDX6m5GTGsF40Pc_R4/lib/python3.11/site-p
ackages/nilearn/image/image.py:1160: UserWarning: Data array used to create
a new image contains 64-bit ints. This is likely due to creating the array w
ith numpy and passing `int` as the `dtype`. Many tools such as FSL and SPM c
annot deal with int64 in Nifti images, so for compatibility the data has bee
n converted to int32.
    return new_img_like(niimg, result, niimg.affine)
```

Seed37 mask



Extract anterior insula (seed37) time series per run and create design matrix

```
In [ ]: # function to find spikes, modified from nltools
def find_spikes(fmri_img, global_spike_cutoff=3, diff_spike_cutoff=3):
    """
    Function to identify global and differential spikes from fMRI Time Series

    Parameters:
        fmri_img: 4D Nifti image of fMRI data (X x Y x Z x T)
        global_spike_cutoff: cutoff to identify spikes in global signal in s
        diff_spike_cutoff: cutoff to identify spikes in average frame differ

    Returns:
        spikes_df: pandas dataframe with spikes as indicator variables
    """

    data = fmri_img.get_fdata()

    global_mn = np.mean(data, axis=(0,1,2))
    frame_diff = np.mean(np.abs(np.diff(data, axis=3)), axis=(0, 1, 2))

    global_outliers = np.append(
        np.where(
            global_mn > np.mean(global_mn) + np.std(global_mn) * global_
        ),
        np.where(
            global_mn < np.mean(global_mn) - np.std(global_mn) * global_
        ),
    )

    frame_outliers = np.append(
        np.where(
            frame_diff
            > np.mean(frame_diff) + np.std(frame_diff) * diff_spike_cuto
```

```

    ),
    np.where(
        frame_diff
        < np.mean(frame_diff) - np.std(frame_diff) * diff_spike_cuto
    ),
)

# build spike regressors
spikes_df = pd.DataFrame(index=range(len(global_mn)))
if global_spike_cutoff is not None:
    for i, loc in enumerate(global_outliers):
        col = f"global_spike{str(i + 1)}"
        spikes_df[col] = 0
        spikes_df.loc[int(loc), col] = 1

# build FD regressors
if diff_spike_cutoff is not None:
    for i, loc in enumerate(frame_outliers):
        col = f"diff_spike{str(i + 1)}"
        spikes_df[col] = 0
        spikes_df.loc[int(loc), col] = 1
return spikes_df

```

```

In [ ]: smoothed_imgs = []
design_matrices = []

for i, (fmri_img, confound_df) in enumerate(zip(fmri_imgs, confounds)):

    # 1. Extract seed time series
    seed_masker = NiftiMasker(mask_img=mask_img,
                               standardize='zscore_sample', # z-score
                               detrend=False, low_pass=None, high_pass=
                               t_r=TR)
    seed_ts = seed_masker.fit_transform(fmri_img)
    seed_ts = seed_ts.mean(axis=1)
    assert seed_ts.shape[0] == confound_df.shape[0], f"Seed time series leng
    assert seed_ts.shape[0] == N_SCANS, f"Seed time series length {seed_ts.s

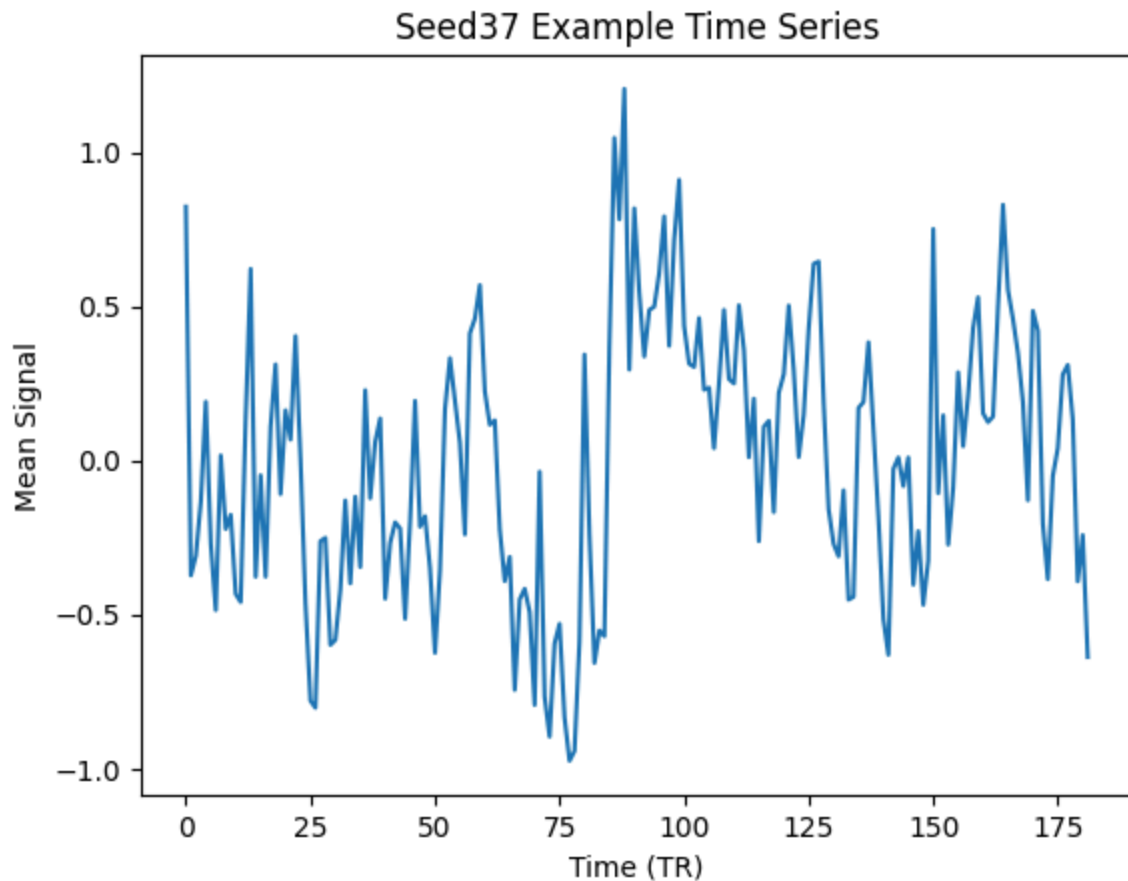
    # 2. Create design matrix for this run
    seed_series = pd.Series(seed_ts, name=f"seed{SEED_LABEL}", index=confound
    poly_df = pd.DataFrame({
        "trend_lin": np.linspace(-1, 1, N_SCANS),
        "trend_quad": np.linspace(-1, 1, N_SCANS) ** 2,
    })
    spikes_df = find_spikes(fmri_img)
    run_dm = pd.concat([seed_series, confound_df, poly_df, spikes_df], axis=
    design_matrices.append(run_dm)

```

```

In [ ]: # plot an example time series
if "dartfs" not in current_dir:
    plt.plot(seed_ts)
    plt.xlabel("Time (TR)")
    plt.ylabel("Mean Signal")
    plt.title(f"Seed{SEED_LABEL} Example Time Series")
    plt.show()

```



Whole-brain GLM

```
In [ ]: # these parameters are included in the design matrix already!
glm = FirstLevelModel(t_r=TR,
                      hrf_model=None,
                      drift_model=None,
                      high_pass=None,
                      standardize=False,
                      noise_model='ar1',
                      minimize_memory=True
)
glm.fit(fmri_imgs, design_matrices=design_matrices)
```


► Parameters

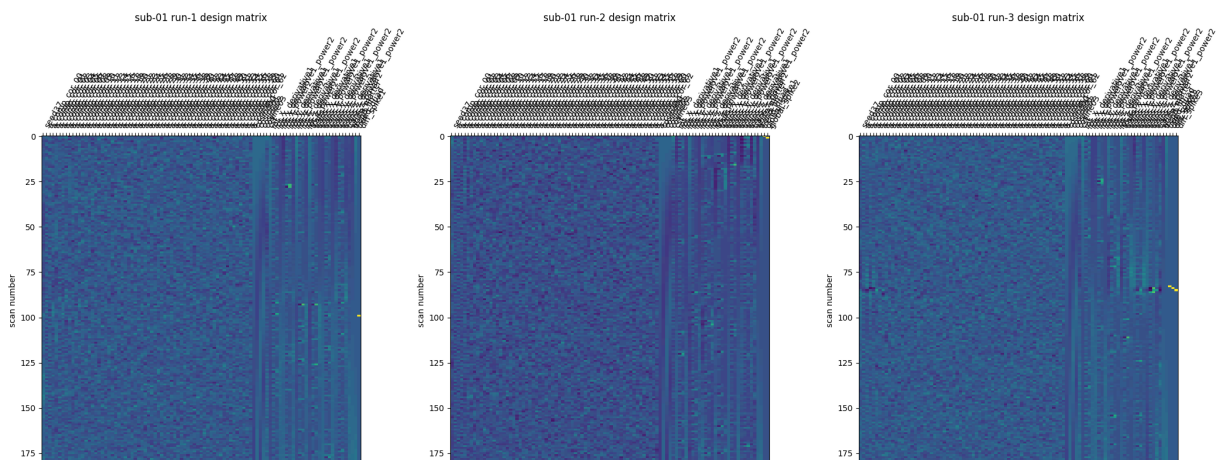
```
In [ ]: if "dartfs" not in current_dir:
        dms = glm.design_matrices_

        fig, axes = plt.subplots(1, len(dms),
                                figsize=(7 * len(dms), 8))

        if len(dms) == 1:
            axes = [axes]

        for _i, dm in enumerate(dms):
            # print(dm.columns)
            plot_design_matrix(dm, axes=axes[_i])
            axes[_i].set_title(f"sub-{sub} run-{_i+1} design matrix", fontsize=12)

        plt.tight_layout()
        plt.savefig(f"design_matrices.png")
        plt.show()
```



Get betas (connectivity map)

```
In [ ]: seed_con_img = glm.compute_contrast(f"seed{SEED_LABEL}", output_type="effect  
seed_con_img.to_filename(f"{save_dir}/sub-{sub}_seed{SEED_LABEL}_connectivity  
print(f"Saved seed connectivity map to {save_dir}/sub-{sub}_seed{SEED_LABEL}
```

```
/var/folders/44/klw90pzs3fz633mk4sk_j1hc0000gn/T/marimo_2486/__marimo__cell_  
Vxnm.py:1: UserWarning: One contrast given, assuming it for all 3 runs  
seed_con_img = glm.compute_contrast(f"seed{SEED_LABEL}", output_type="effect_size")
```

Saved seed connectivity map to /Users/gabe/Downloads/exam/connectivity/output/sub-01/sub-01_seed37_connectivity_beta.nii.gz

```
In [ ]: if "dartfs" not in current_dir:  
plot_stat_map(seed_con_img, title=f"sub-{sub} seed{SEED_LABEL} connectivity  
plt.show()
```

sub-01 seed37 connectivity map (beta values)

